

Correction — TP K-means

34. Niveau 1 — Jeu de données de 15 clients

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

data = pd.DataFrame({
    "revenu_annuel": [15, 18, 20, 22, 25, 18, 22, 25, 28, 30, 75, 80, 85, 88, 90],
    "score_depense": [12, 20, 15, 25, 18, 78, 85, 80, 92, 75, 82, 88, 95, 80, 90],
})
features = ["revenu_annuel", "score_depense"]
```

Modèles avec K = 2 et K = 3

```
model_k2 = KMeans(n_clusters=2, random_state=42, n_init=10)
data["cluster_k2"] = model_k2.fit_predict(data[features])

model_k3 = KMeans(n_clusters=3, random_state=42, n_init=10)
data["cluster_k3"] = model_k3.fit_predict(data[features])

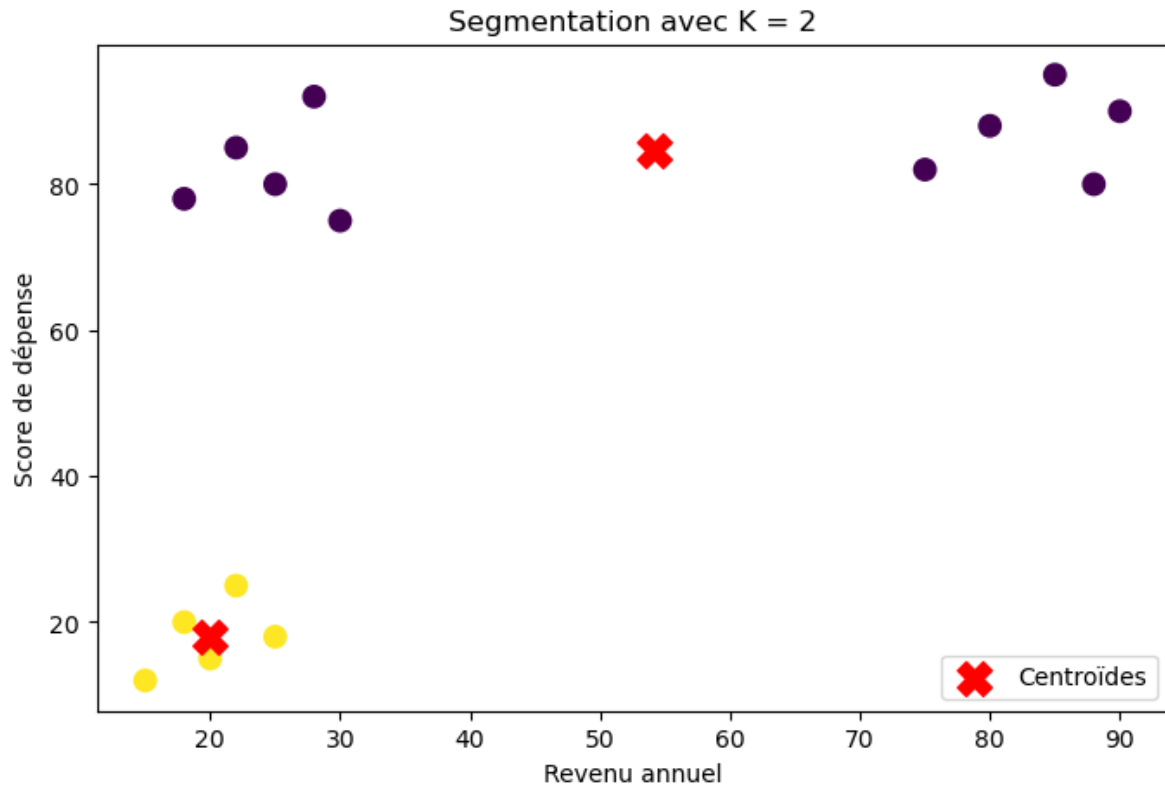
print("Inertie K = 2 :", model_k2.inertia_)
print("Inertie K = 3 :", model_k3.inertia_)
print("Centroïdes K = 2 :\n", model_k2.cluster_centers_)
print("Centroïdes K = 3 :\n", model_k3.cluster_centers_)
```

```
Inertie K = 2 : 9487.4
Inertie K = 3 : 722.4
Centroïdes K = 2 :
```

```
[[54.1 84.5]
 [20.  18. ]]
Centroides K = 3 :
[[24.6 82. ]
 [83.6 87. ]
 [20.  18. ]]
```

Les numéros de clusters sont arbitraires : le cluster 0 ne représente pas nécessairement le profil ayant le revenu le plus faible.

```
plt.figure(figsize=(8, 5))
plt.scatter(
    data["revenu_annuel"],
    data["score_depense"],
    c=data["cluster_k2"],
    cmap="viridis",
    s=80,
)
plt.scatter(
    model_k2.cluster_centers[:, 0],
    model_k2.cluster_centers[:, 1],
    c="red",
    marker="X",
    s=200,
    label="Centroides",
)
plt.xlabel("Revenu annuel")
plt.ylabel("Score de dépense")
plt.title("Segmentation avec K = 2")
plt.legend()
plt.show()
```



L'inertie baisse nécessairement lorsque K augmente : chaque observation dispose de davantage de centroïdes possibles et la somme des distances au centroïde le plus proche ne peut donc qu'être plus faible ou égale. Cette baisse ne prouve pas que le modèle est meilleur ; avec un cluster par observation, l'inertie serait nulle.

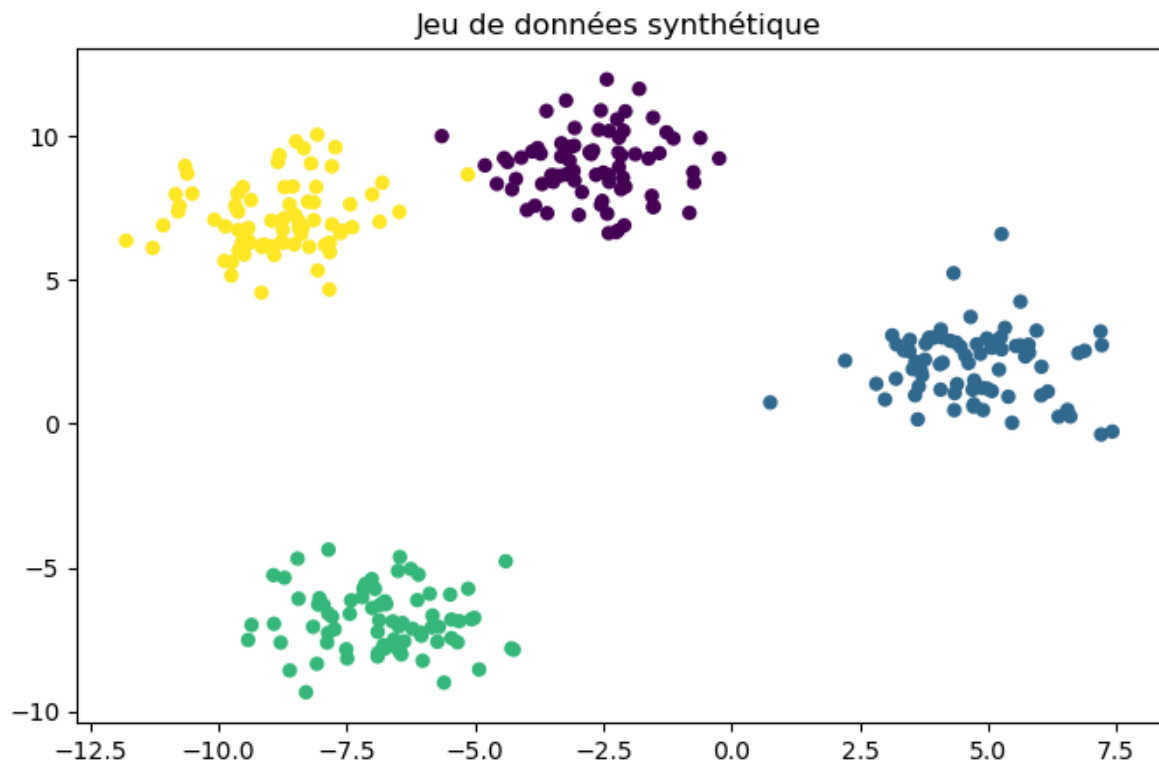
35. Niveau 2 — Jeu synthétique

```
from sklearn.datasets import make_blobs

X, y_true = make_blobs(
    n_samples=300,
    centers=4,
    cluster_std=1.2,
    random_state=42,
)

plt.figure(figsize=(8, 5))
plt.scatter(X[:, 0], X[:, 1], c=y_true, cmap="viridis", s=25)
```

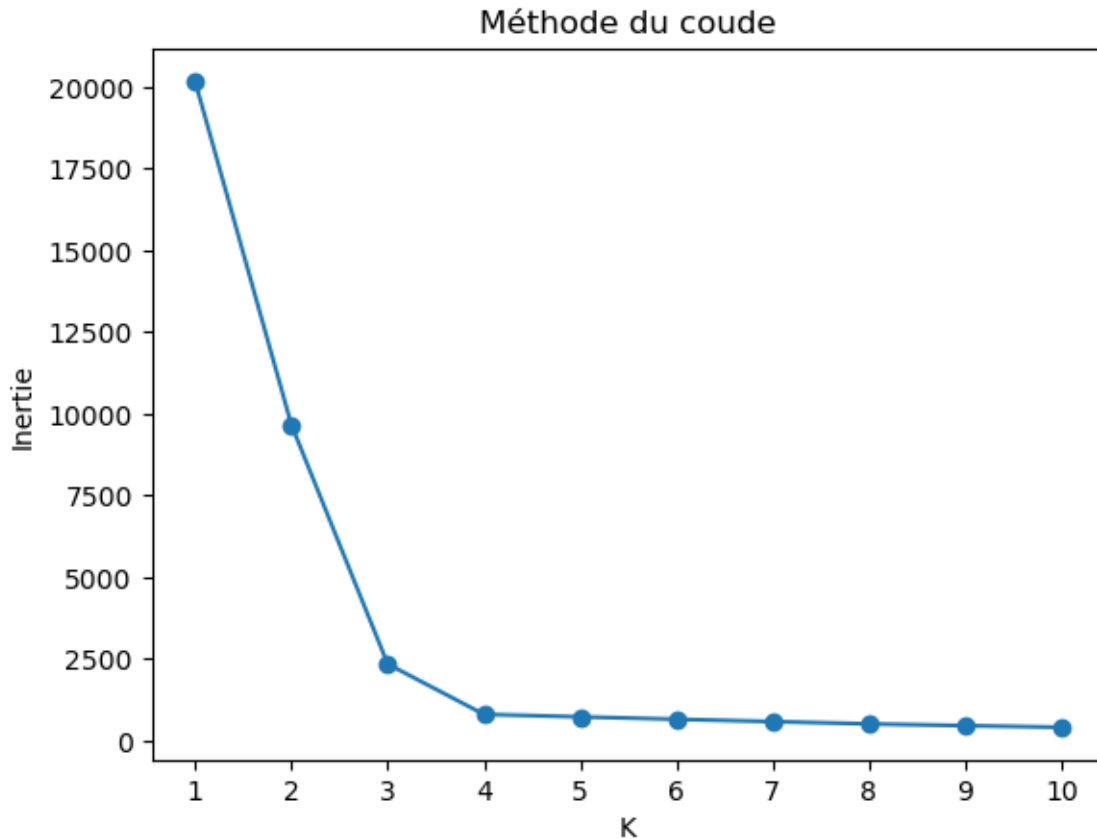
```
plt.title("Jeu de données synthétique")
plt.show()
```



```
inertias = []
k_values = range(1, 11)

for k in k_values:
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    model.fit(X)
    inertias.append(model.inertia_)

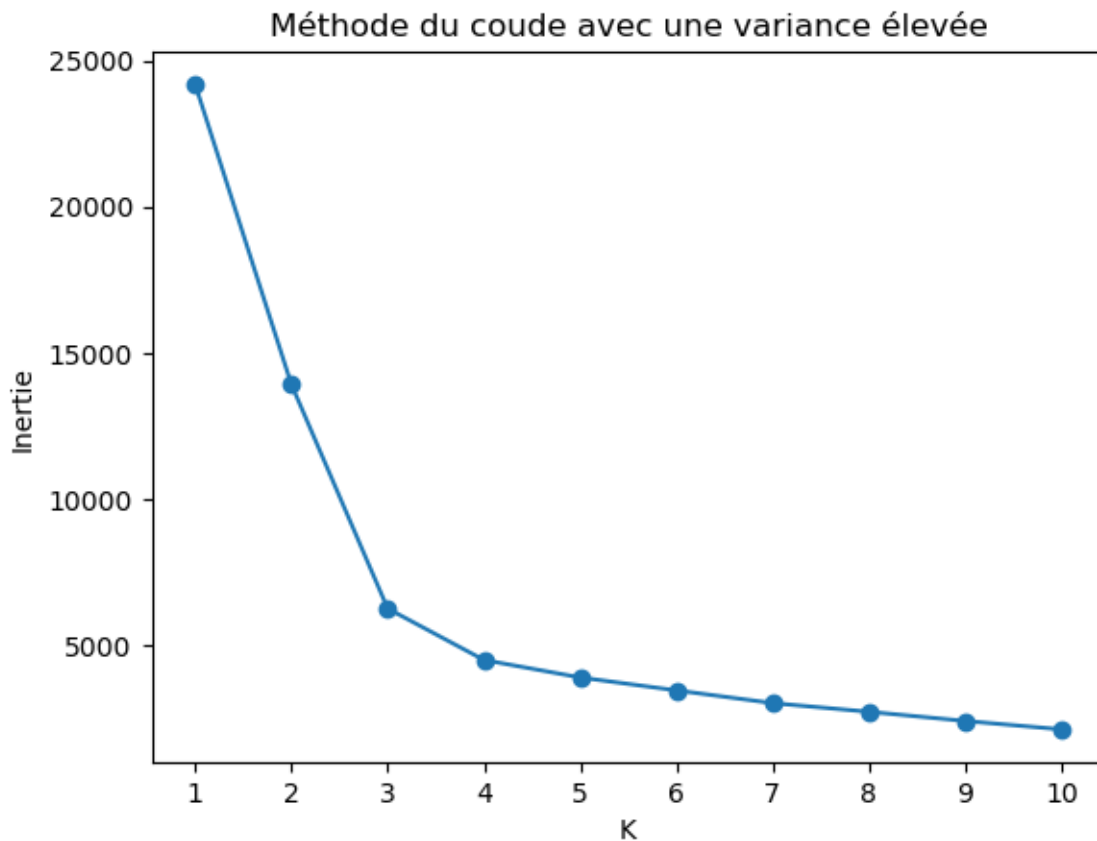
plt.plot(k_values, inertias, marker="o")
plt.xlabel("K")
plt.ylabel("Inertie")
plt.title("Méthode du coude")
plt.xticks(list(k_values))
plt.show()
```



Le coude est généralement visible autour de $K = 4$, ce qui correspond aux quatre centres utilisés pour générer les données. Avec `cluster_std=3.0`, les groupes se recouvrent davantage et le coude devient moins net.

```
X_noisier, y_noisier = make_blobs(  
    n_samples=300,  
    centers=4,  
    cluster_std=3.0,  
    random_state=42,  
)  
  
inertias_noisier = []  
for k in k_values:  
    model = KMeans(n_clusters=k, random_state=42, n_init=10)  
    model.fit(X_noisier)  
    inertias_noisier.append(model.inertia_)  
  
plt.plot(k_values, inertias_noisier, marker="o")
```

```
plt.xlabel("K")
plt.ylabel("Inertie")
plt.title("Méthode du coude avec une variance élevée")
plt.xticks(list(k_values))
plt.show()
```



36. Niveau 3 — Segmentation e-commerce

Charger et inspecter les données

```
import pandas as pd

df_challenge = pd.read_csv("data/ecommerce_segmentation.csv")
df_challenge.head()
```

```
df_challenge.describe()
df_challenge.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 81 entries, 0 to 80
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  -
0   age                   81 non-null    int64
1   revenu_annuel         81 non-null    float64
2   score_depense         81 non-null    float64
3   frequence_achats      81 non-null    int64
4   anciennete_mois       81 non-null    int64
5   segment               81 non-null    object
dtypes: float64(2), int64(3), object(1)
memory usage: 3.9+ KB
```

La colonne `segment` est une information connue lors de la génération du jeu de données. Elle doit être exclue du clustering, car K-means est utilisé ici de manière non supervisée.

```
features = [
    "age",
    "revenu_annuel",
    "score_depense",
    "frequence_achats",
    "anciennete_mois",
]
X_challenge = df_challenge[features]
```

Pipeline et comparaison des valeurs de K

```
from sklearn.cluster import KMeans
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt

def make_pipeline(k):
```

```

return Pipeline([
    ("scaler", StandardScaler()),
    ("kmeans", KMeans(n_clusters=k, random_state=42, n_init=10)),
])

metrics = []
for k in range(2, 9):
    pipeline = make_pipeline(k)
    labels = pipeline.fit_predict(X_challenge)
    X_scaled = pipeline.named_steps["scaler"].transform(X_challenge)
    model = pipeline.named_steps["kmeans"]

    metrics.append({
        "k": k,
        "inertia": model.inertia_,
        "silhouette": silhouette_score(X_scaled, labels),
    })

metrics = pd.DataFrame(metrics)
metrics

```

	k	inertia	silhouette
0	2	122.261833	0.596113
1	3	100.804562	0.446875
2	4	81.864037	0.472009
3	5	65.607188	0.315141
4	6	58.779695	0.308150
5	7	56.034535	0.283802
6	8	50.081952	0.304453

```

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

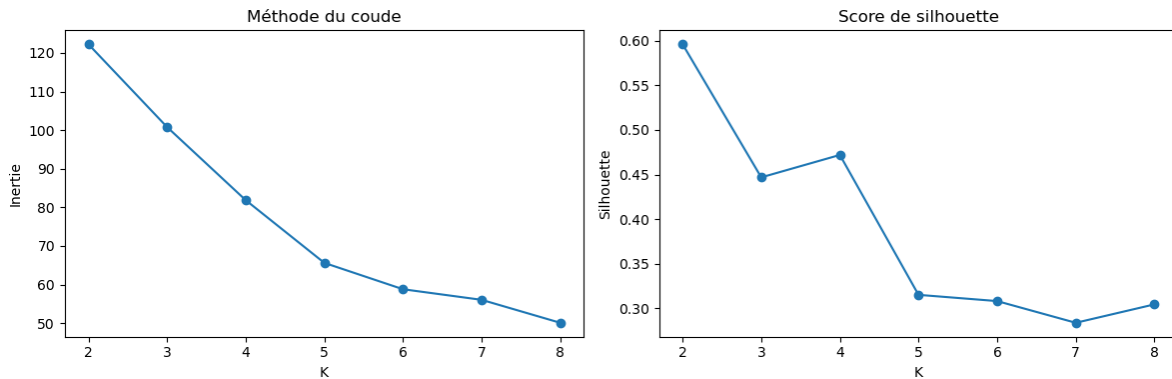
axes[0].plot(metrics["k"], metrics["inertia"], marker="o")
axes[0].set_title("Méthode du coude")
axes[0].set_xlabel("K")
axes[0].set_ylabel("Inertie")

axes[1].plot(metrics["k"], metrics["silhouette"], marker="o")
axes[1].set_title("Score de silhouette")
axes[1].set_xlabel("K")

```

```
axes[1].set_ylabel("Silhouette")
```

```
plt.tight_layout()
plt.show()
```



L'inertie diminue toujours quand K augmente. Le score de silhouette est utile pour comparer la compacité des groupes et leur séparation. On retient ici la valeur qui maximise la silhouette, après vérification que le choix reste cohérent avec le coude et l'interprétation métier.

```
best_k = int(metrics.loc[metrics["silhouette"].idxmax(), "k"])
print("K retenu :", best_k)
```

K retenu : 2

Modèle final et interprétation métier

```
final_pipeline = make_pipeline(best_k)
df_result = df_challenge.copy()
df_result["cluster"] = final_pipeline.fit_predict(X_challenge)

cluster_means = (
    df_result.groupby("cluster")[features].mean().round(1)
)
cluster_sizes = df_result["cluster"].value_counts().sort_index()

print("Taille des clusters :")
print(cluster_sizes)
print("\nMoyennes dans les unités d'origine :")
print(cluster_means)
```

```
Taille des clusters :
cluster
0      40
1      41
Name: count, dtype: int64
```

```
Moyennes dans les unités d'origine :
      age  revenu_annuel  score_depense  frequence_achats  anciennete_mois
cluster
0      52.1           94.4           79.5             36.0             68.2
1      34.8           35.5           28.6              6.2             11.4
```

L'interprétation doit partir du tableau `cluster_means`, et non du numéro du cluster. Selon les moyennes observées, on pourra par exemple identifier :

- des clients occasionnels ou à faible engagement : revenu, score, fréquence et ancienneté faibles ;
- des clients premium fidèles : revenu, score, fréquence et ancienneté élevés ;
- de nouveaux clients prometteurs : score et fréquence élevés, mais ancienneté faible ;
- un profil atypique à vérifier, si un cluster est très petit ou dominé par une observation extrême.

La standardisation rend les variables comparables pour calculer les distances, mais elle transforme leurs unités en écarts-types. Une moyenne standardisée est donc difficile à interpréter pour un métier. Il faut conserver les labels obtenus avec les données standardisées, puis calculer les moyennes des clusters sur les variables originales : âge en années, revenu dans son unité, fréquence en achats et ancienneté en mois.

37. À retenir

- L'inertie diminue mécaniquement quand le nombre de clusters augmente.
- La méthode du coude et le score de silhouette sont complémentaires.
- La standardisation évite qu'une variable domine les distances par sa seule échelle.
- Les profils métier doivent être décrits à partir des données non standardisées.